

Xor[a, b]

Out[*n*]= $a \vee b$

Tower of Hanoi Visualization

ColorData[1, "ColorList"]

Out[*n*]= {

ColorData[2, "ColorList"]

Out[*n*]= {

ColorData[3, "ColorList"]

Out[*n*]= {

ColorData[3][2]

Out[*n*]= 

ColorData[4, "ColorList"]

Out[*n*]= {

ColorData[5, "ColorList"]

Out[*n*]= {

```
In[n]:= THStack[ls_] := MapIndexed[{ColorData[3][#],  
  Rectangle[{-#, Length[ls] - #2[[1]]}, {#, 1 + Length[ls] - #2[[1]]}]} &, ls]
```

```
In[n]:= THStack[x0_] := MapIndexed[{ColorData[3][#],  
  Rectangle[{x0 - #, Length[ls] - #2[[1]]}, {x0 + #, 1 + Length[ls] - #2[[1]]}]} &, ls]
```

```
In[n]:= THTopStack[ls_, center_] := Map[{ColorData[3][#], Disk[center, #]} &, Reverse[ls]]
```

```
In[n]:= THSide[{a_, b_, c_}, opts___] :=  
  Graphics[{EdgeForm[LightGray], THStack[a, -17], THStack[b, 0],  
    THStack[c, 17], {Brown, Line[Table[{x, 0}, {x, 8}], {x, {-17, 0, 17}}]}],  
  opts, PlotRange → {{-33, 33}, {0, 8}}]
```

```
In[n]:= THTop[{a_, b_, c_}, opts___] :=  
  Graphics[{EdgeForm[LightGray], THTopStack[a, {-17, 0}], THTopStack[b, {0, 0}],  
    THTopStack[c, {17, 0}], {Brown, Point[Table[{x, 0}, {x, {-17, 0, 17}}]}],  
  opts, PlotRange → {{-33, 33}, {-16, 16}}]
```

```
In[n]:= THTopA[{a_, b_, c_}, opts___] :=  
  Graphics[{EdgeForm[LightGray], THTopStack[a, {-8, -4.62}],  
    THTopStack[b, {0, 9.238}], THTopStack[c, {8, -4.62}],  
    {Brown, Point[{{-8, -4.62}, {0, 9.238}, {8, -4.62}}]}], opts, PlotRange → 16]
```

16. / Sqrt[3] CirclePoints[3]

Out[*n*]= {{8., -4.6188}, {0., 9.2376}, {-8., -4.6188}}

THSide[{{1, 2, 3, 4, 5, 6, 7, 8}, {}, {}]}

Out[«]=



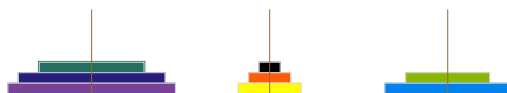
THTop[{{1, 2, 3, 4, 5, 6, 7, 8}, {}, {}]}

Out[«]=



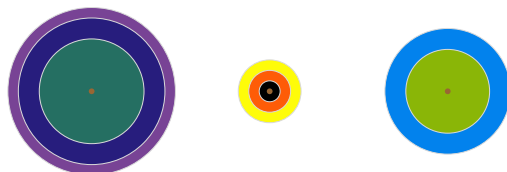
THSide[{{5, 7, 8}, {1, 2, 3}, {4, 6}}]

Out[«]=

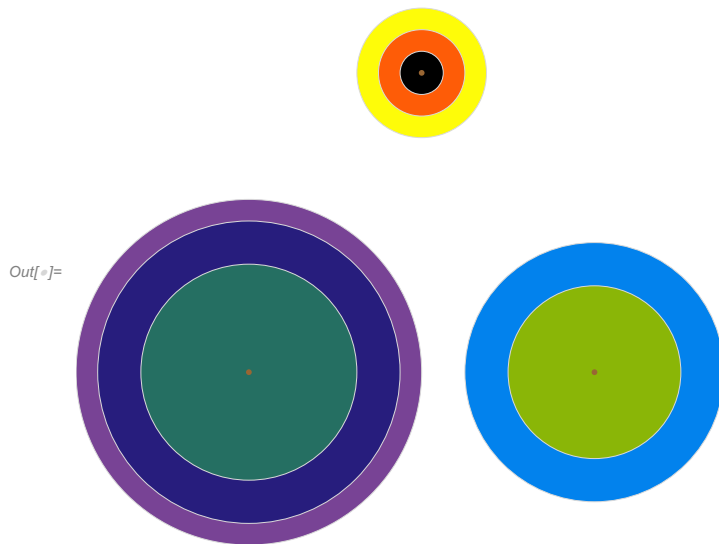


THTop[{{5, 7, 8}, {1, 2, 3}, {4, 6}}]

Out[«]=



THTopA[{{5, 7, 8}, {1, 2, 3}, {4, 6}}]



```
In[ ]:= THMove[abc_, {x_, y_}] :=
  If[Length[abc[[x]]] > 0 && (Length[abc[[y]]] == 0 || (abc[[x, 1]] < abc[[y, 1]])),
    Insert[Delete[abc, {x, 1}], abc[[x, 1]], {y, 1}], abc]
```

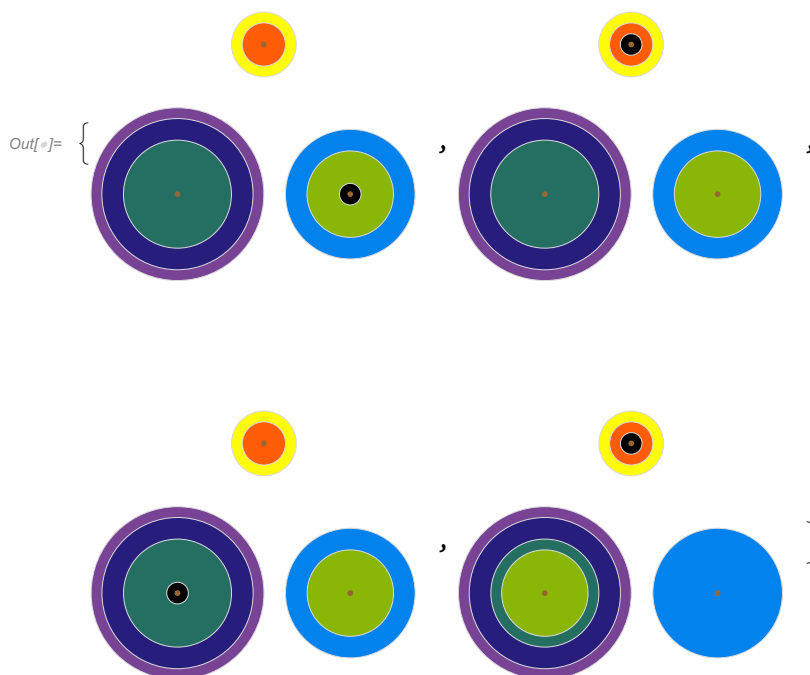
```
THMove[{{5, 7, 8}, {1, 2, 3}, {4, 6}}, {3, 1}]
```

Out[]= {{4, 5, 7, 8}, {1, 2, 3}, {6}}

```
Union[THMove[{{5, 7, 8}, {1, 2, 3}, {4, 6}}, #] & /@
  Select[Tuples[{1, 2, 3}, 2], Apply[UnsameQ]]]
```

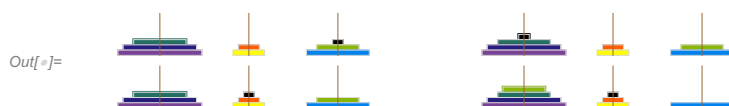
Out[]= {{{5, 7, 8}, {2, 3}, {1, 4, 6}}, {{5, 7, 8}, {1, 2, 3}, {4, 6}},
 {{1, 5, 7, 8}, {2, 3}, {4, 6}}, {{4, 5, 7, 8}, {1, 2, 3}, {6}}}

THTopA /@ %

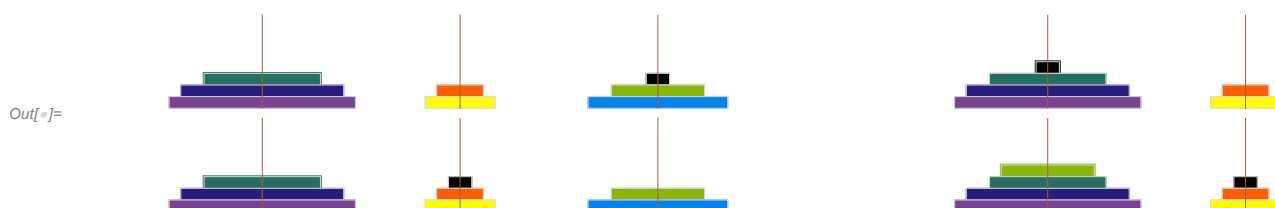


```
In[ ]:= AllTHMoves[abc_] := Union[THMove[abc, #] & /@ Select[Tuples[{1, 2, 3}, 2], Apply[UnsameQ]]]
```

```
Multicolumn[THSide /@ AllTHMoves[{{5, 7, 8}, {1, 2, 3}, {4, 6}}]]
```



```
Multicolumn[THSide[#, ImageSize -> 400] & /@ AllTHMoves[{{5, 7, 8}, {1, 2, 3}, {4, 6}}]]
```



Recursion

How many steps does it take?

This is going to be a number so it will be amenable to recursive formulas.

We don't even know if there is a solution. Even if we believed there was we don't know how. This comes first.

After that we want to get a recursive formula. In this case the formula will be

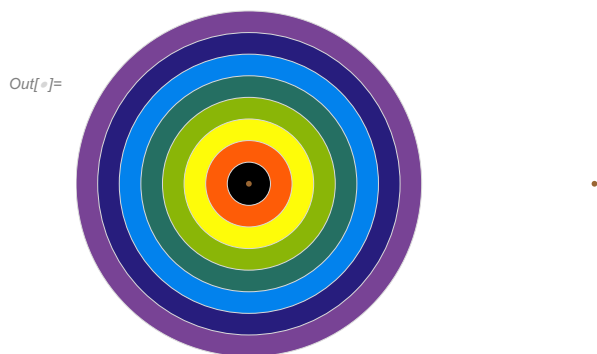
$$T(n) = 2 T(n-1) + 1$$

After getting this we will want to get a closed form solution to the recursive formula.

Steps to Solving a Problem

Clear Statement of the Problem

Start with all eight disks on one peg. Each disk has a different size. We may assume the radii are 1, 2, 3, 4, 5, 6, 7, 8



At each turn we can move a disk from the top of one stack to the top of another stack under a condition. The condition that lets us move a disk is that we must never put a larger disk on a smaller disk.

Consider Small Cases

The smaller the better.

We don't just say 8 is too many, how about 7?

For the Tower of Hanoi. the simplest case is no disks.

Out[]:=

There is no task with $n = 0$ so we can say the steps to completion is $T(0) = 0$

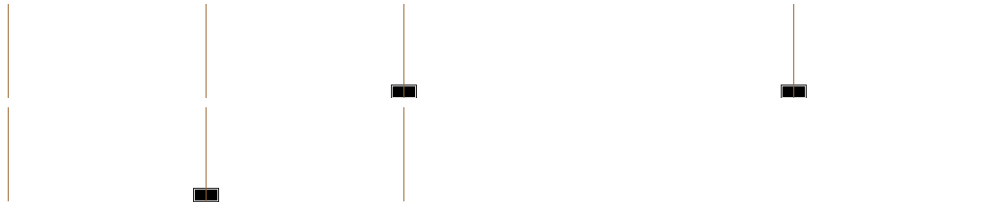
Next is $n = 1$

Out[]:=

The task is to move this disk to another peg.

The allowed moves include moving it to another peg because the condition on moving disks is vacuous.

Out[]:=



So there is one turn and the task is complete so $T(1) = 1$

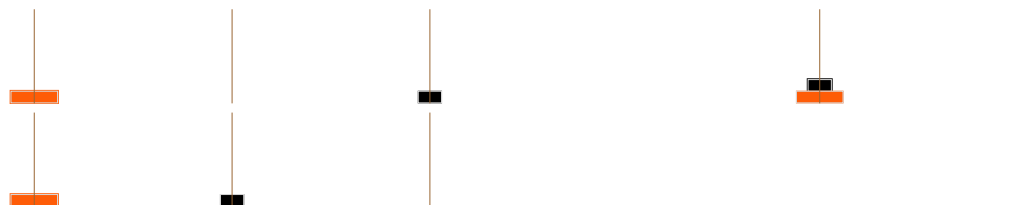
Next is $n = 2$

Out[]:=



It might not be immediately clear so we enumerate all possible moves.

Out[]:=



There is symmetry so there is essentially one choice. Move the top disk to another peg.



Out[]:=



Now there are three choices of moves.

Out[]:=



Two moves don't get us anywhere. We can move back to the initial condition or move the smallest disk to the other empty peg, which we could have done on the last step.

The new step is moving the largest disk to the empty peg. Before deciding if we are making progress let's consider it.

Out[]:=

Out[]:=

We see that one of the moves now completes the task.

Out[]:=

How did it work?

We were able to move the largest disk after getting the small one off of it and having an empty peg to move it to.

We can only move the largest disk to an empty peg. That's a direct consequence of the rules. We can call that a lemma. It could have been part of the rules without changing the game.

Identify Lemmas

Moving on to $n = 3$, let's see if we can make an educated guess.

We know that we can't move the largest disk until all the disks on top of it have been moved somewhere and there is an empty peg.

Out[]:=



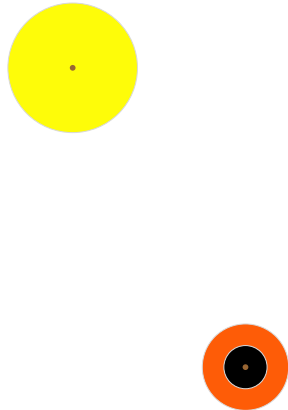
Visualization helps.

This is exactly our solution to $n=2$ except the third disk is still remaining on the original peg.

There is only one empty peg to move the third disk.

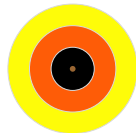
Let's visualize what happens after moving the third disk.

Out[]:=



The situation is symmetric. We can complete the task by moving the smaller disks back onto the larger disk.

Out[]:=



How many steps did that take?

We moved two disks in $T(2)$ turns. Then we moved the larger disk on one turn. Then we moved two disks in $T(2)$ turns.

$$T(2) + 1 + T(2) = 2 T(2) + 1$$

This formula is something we want to generalize, and we will want to justify that generalization.

We can also immediately conclude that $T(3) = 7$

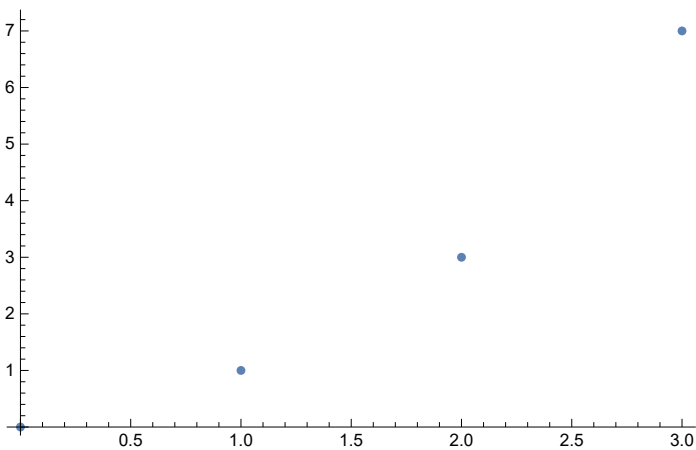
Collect what we know so far

Out[]=

n	T(n)
0	0
1	1
2	3
3	7

ListPlot[{0, 1, 3, 7}, DataRange → {0, 3}]

Out[]=



Attempt: Try a recursive formula that generalizes what you know

Moved $n-1$ disks in $T(n-1)$ turns. Then we move the largest disk on one turn. Then move $n-1$ disks in $T(n-1)$ turns.

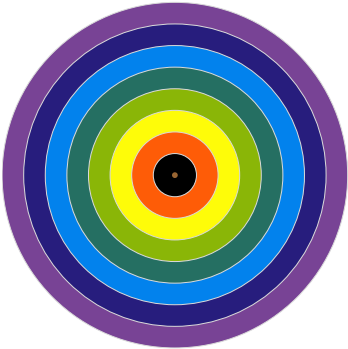
$$T(n-1) + 1 + T(n-1) = 2 T(n-1) + 1$$

$$T(n) = 2 T(n-1) + 1$$

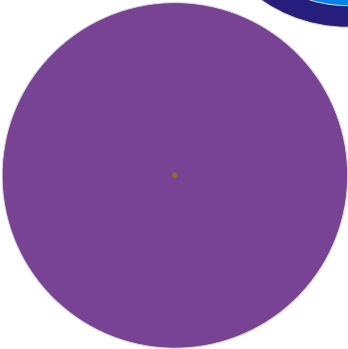
Prove the recursive method works

Moved $n-1$ disks in $T(n-1)$ turns. Then we move the largest disk on one turn. Then move $n-1$ disks in $T(n-1)$ turns.

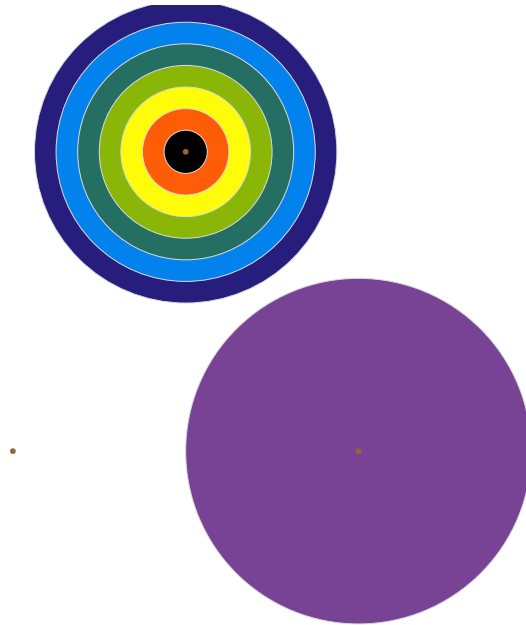
Out[]=



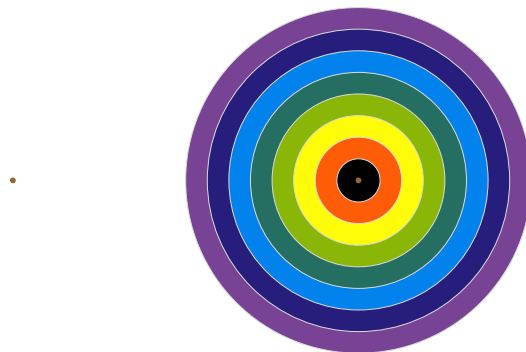
Out[]=



Out[]:=



Out[]:=



We need a valid method for proving this. We will need induction.

Solve the Recurrence for a closed form

$$T(n) = 2 T(n-1) + 1$$

Need an initial condition. Use $T(0) = 0$.

Compare with things you've seen

This is like exponential growth.

Experiment

Look at the values and guess

Out[]:=

n	T(n)
0	0
1	1
2	3
3	7
4	15
5	31
6	63
7	127
8	255
9	511
10	1023

Out[]:=

n	T(n)	2^n
0	0	1
1	1	2
2	3	4
3	7	8
4	15	16
5	31	32
6	63	64
7	127	128
8	255	256
9	511	512
10	1023	1024

Use algebra and formal manipulations

Suppose

$$f(n) = T(n) + 1$$

then $f(n)$ satisfies

$$f(n) = T(n) + 1 = (2 T(n-1) + 1) + 1 = 2(f(n-1) - 1) + 2 = 2 f(n-1)$$

$$\text{i.e., } f(n) = 2 f(n-1)$$

and since $f(0) = T(0) + 1 = 0 + 1 = 1$ we get the solution.